

Perfect 😊 Let's finish 7.11 with one last concept: **Origin / Referer validation**.

7.11.4 — Origin & Referer Validation

We've already seen that the server can ask:

"Is this request actually coming from my application?"

One way to help answer that is to inspect the browser's request headers.

1. The `Origin` header

Suppose your legitimate frontend is:

```
https://myapp.com
```



and it sends:

```
POST /api/transfer  
Origin: https://myapp.com
```

The server can check:

```
Is Origin == https://myapp.com ?
```



If yes:

```
✅ Expected origin
```



If an attacker tries to initiate the request from:

```
https://evil.com
```



the browser may send:

```
POST /api/transfer  
Origin: https://evil.com
```

Your server can reject it:

```
✖ Unexpected origin
```



2. What does **Origin** actually tell us?

Think of it as:

"Which origin initiated this request?"

For example:

```
Origin: https://app.example.com
```

means the request originated from that web origin.

Remember our definition:

```
Origin =  
scheme + host + port
```



So:

```
https://app.example.com
```



is an origin.

And:

```
https://evil.com
```



is a completely different origin.

3. Example in a Next.js Route Handler

Suppose:

app/api/transfer/route.ts



You could conceptually do:

```
export async function POST(request: Request) {
  const origin = request.headers.get("origin");

  if (origin !== "https://myapp.com") {
    return Response.json(
      { error: "Invalid origin" },
      { status: 403 }
    );
  }

  // Continue...
}
```

So:

https://myapp.com



→ allowed.

https://evil.com



→ rejected.

4. What about **Referer**?

There's another browser header:

Referer: https://myapp.com/transfer

It can tell the server the URL of the page that led to the request.

For example:

```
User is on:  
https://myapp.com/transfer
```



↓

```
POST /api/transfer
```

```
Referer:  
https://myapp.com/transfer
```

The server can inspect it.

But there's an important difference:

Origin

Usually gives:

```
https://myapp.com
```



Referer

Can give something like:

```
https://myapp.com/transfer
```



So **Referer** contains more detailed URL information.

5. Why prefer **Origin** when available?

For CSRF protection, you're generally interested in:

Which origin made this request?

You don't necessarily care which exact page.

So:

```
Origin:  
https://myapp.com
```



is cleaner than:

Referer:
https://myapp.com/transfer



Also, browsers and privacy settings can suppress or reduce **Referer** information.

So applications shouldn't blindly assume **Referer** will always be present.

6. How does this stop CSRF?

Let's put everything together.

Legitimate request

User is on:

https://myapp.com



They click:

Transfer ₹500



Browser sends:

```
POST /api/transfer
Origin: https://myapp.com
Cookie: sessionId=abc123
X-CSRF-Token: xyz789
```

Server:

```
Origin?
  ↓
https://myapp.com
  ↓
☒

Session?
  ↓
valid
  ↓
☒

CSRF token?
  ↓
```



valid



Authorization?



allowed



Perform transfer

Attacker request

User visits:

`https://evil.com`



Attacker tries to make:

POST `https://myapp.com/api/transfer`
Origin: `https://evil.com`

Server checks:

Origin?



`https://evil.com`



Request rejected:

403 Forbidden

7. Now compare the defenses

You've now learned several layers.

HttpOnly

JavaScript



✗ Can't read session cookie



SameSite

Cross-site request



Browser decides whether
cookie can accompany it



CSRF token

Request



Does it contain the expected
CSRF proof?



Origin validation

Request



Did it come from an allowed origin?



CORS

Cross-origin JavaScript



Can it READ the response?



Notice how these are **not interchangeable**.



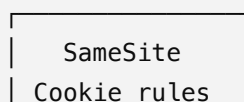
The complete mental model

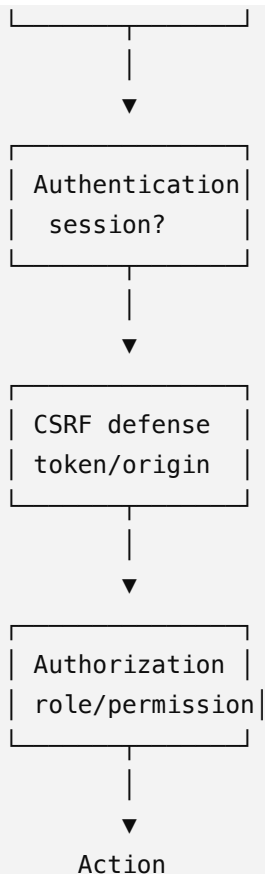
Imagine:

Browser



| Request





And **CORS** sits somewhat separately:

Cross-origin request

CORS

Can browser JavaScript
read the response?



🎉 7.11 is now complete

You've covered:

- ☒ CSRF attack
- ☒ Why cookie authentication is relevant to CSRF
- ☒ HttpOnly
- ☒ SameSite=Lax
- ☒ SameSite=Strict
- ☒ SameSite=None
- ☒ Same-site vs same-origin
- ☒ CORS

-  CORS preflight
-  `credentials: "include"`
-  Bearer vs cookie authentication
-  CSRF tokens
-  Origin / Referer validation

So now we're moving to:

7.12 — Middleware / Proxy Authentication

This is where you'll learn something really practical:

Instead of putting `auth()` checks manually inside every protected page, can we intercept requests before they reach those pages?

We'll see **exactly what Middleware/Proxy does**, what it should and shouldn't protect, and how it differs from the `auth()` checks we've already written.



yes okay

Awesome 😄 Now we're moving into **7.12 — Middleware / Proxy Authentication**. This one is especially useful for interviews because people often confuse **middleware protection** with **actual authorization**.

7.12 — Middleware / Proxy Authentication

Let's start with the problem.

Suppose your application has:

```
/dashboard
/settings
/orders
/profile
/admin
```



And all of these require login.

Without a centralized request guard, you might write:

```
</> TypeScript
```



```
// dashboard/page.tsx
const session = await auth();

if (!session) redirect("/login");
```

Then again:

```
</> TypeScript

// settings/page.tsx
const session = await auth();

if (!session) redirect("/login");
```

And again:

```
</> TypeScript

// orders/page.tsx
const session = await auth();

if (!session) redirect("/login");
```

That's repetitive.

So you might think:

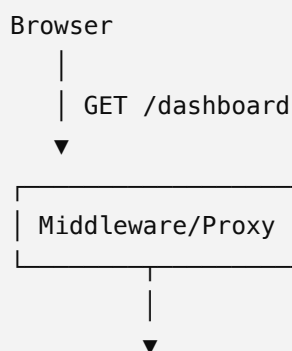
"Can I intercept the request before Next.js renders the page?"

Yes.

That's where the middleware/proxy layer comes in.

1. What is Middleware/Proxy?

Conceptually:



```
/dashboard  
page.tsx
```

Instead of:

```
Browser  
  ↓  
dashboard/page.tsx  
  ↓  
auth()  
  ↓  
check session
```



you can have:

```
Browser  
  ↓  
Middleware/Proxy  
  ↓  
check authentication  
  ↓  
allow / redirect  
  ↓  
dashboard/page.tsx
```



So Middleware/Proxy acts like a **checkpoint before the request reaches the destination**.

2. Very simple example

Conceptually, you could have:

```
</> TypeScript  
  
export { auth as proxy } from "@auth";
```



with a matcher such as:

```
</> TypeScript  
  
export const config = {  
  matcher: ["/dashboard/:path*", "/settings/:path*"],  
};
```



The exact recommended Auth.js/Next.js setup can vary by version, but the important architecture is:

```
/dashboard/*  
  ↓  
Proxy/Middleware  
  ↓  
Authentication check  
  ↓  
Authenticated? — No —> /login  
  |  
  Yes  
  ↓  
Dashboard
```



3. What does `matcher` mean?

This part is important.

Suppose:

```
</> TypeScript  
  
export const config = {  
  matcher: ["/dashboard/:path*"],  
};
```



You're basically saying:

"Run this request guard for URLs under `/dashboard`."

So:

```
/dashboard
```



matches.

```
/dashboard/settings
```



matches.

```
/dashboard/orders/123
```



matches.

But:

/about



doesn't.

4. What happens when the user visits /dashboard ?

Imagine the user isn't logged in.

Browser:

```
</> http
```



```
GET /dashboard
```

Request reaches Proxy/Middleware:

Is there a valid authenticated session?



Answer:

✗ No



So it redirects:

```
/dashboard
```

↓

```
/login
```



The page itself doesn't even need to render the dashboard.

5. What if the user IS logged in?

Browser:

```
</> http
```



```
GET /dashboard
```

```
Cookie: session=...
```

Proxy:

Valid session?

↓



```
YES
↓
Allow request
```

Then:

```
/dashboard
↓
page.tsx
↓
Dashboard rendered
```



6. But here's the BIG question:

You might be thinking:

"If Middleware/Proxy already checks authentication, why did we learn `auth()` inside pages and APIs?"

🔥 **Excellent question.**

Because Proxy/Middleware should **not be your only security boundary**.

Consider:

```
Browser
↓
Proxy
↓
Dashboard page
```



That's useful for protecting the page navigation.

But your API might be:

```
POST /api/users/123/delete
```



A malicious user doesn't have to click your UI.

They can directly call:

```
</> Bash
curl -X DELETE https://myapp.com/api/users/123
```



So your API should still perform its own authentication/authorization checks.

For example:

</> TypeScript



```
export async function DELETE() {
  const session = await auth();

  if (!session) {
    return Response.json(
      { error: "Unauthorized" },
      { status: 401 }
    );
  }

  if (session.user.role !== "ADMIN") {
    return Response.json(
      { error: "Forbidden" },
      { status: 403 }
    );
  }

  // delete user
}
```

7. Think of Proxy as the building entrance 🏢

Imagine your application is an office building.

Proxy/Middleware

The security guard at the entrance:

"Are you logged in?"



Page

The room you're trying to enter:

Dashboard



API

Another door inside the building:

Delete User



You wouldn't say:

"The guard checked me at the front door, so every internal door should blindly trust me."

Sensitive doors can still check:

Who are you?
Are you allowed to do this?



That's why:

Proxy
↓
coarse request protection

auth() inside server code
↓
actual security boundary for sensitive operations

authorization
↓
what this user is allowed to do



8. Authentication vs Authorization here

This is another place people get confused.

Proxy might check:

Is user logged in?



That's:

Authentication

But suppose:

/admin/users



Only admins can access it.

You need:

```
</> TypeScript
```



```
if (session.user.role !== "ADMIN") {  
  // reject  
}
```

That's:

Authorization

So you can have:

```
Proxy
```

```
↓
```

```
Is authenticated?
```

```
↓
```

```
YES
```

```
↓
```

```
Admin page
```

```
↓
```

```
Is role ADMIN?
```

```
↓
```

```
YES
```

```
↓
```

```
Allow
```



9. Should Proxy check every permission?

Usually, you don't want to put all your business authorization logic into one giant Proxy/Middleware function.

Imagine:

```
users:read  
users:create  
users:update  
users:delete  
orders:read  
orders:create  
orders:refund  
products:write  
reports:view  
...
```



You could technically create an enormous request guard, but that becomes difficult to maintain.

Instead, keep sensitive authorization close to the operation.

For example:

`</>` TypeScript

```
export async function DELETE() {
  const session = await auth();

  if (!session) {
    return unauthorized();
  }

  if (!session.user.permissions.includes("users:delete")) {
    return forbidden();
  }

  // delete
}
```

That's much clearer.

10. Another important thing: Proxy isn't magic security

Suppose your Proxy protects:

`/dashboard`

But your API is:

`/api/account`

and you forget to protect the API.

A user might not be able to navigate to the dashboard, but they could potentially directly call:

`/api/account`

So don't think:

Proxy protects dashboard

↓

Entire application is protected

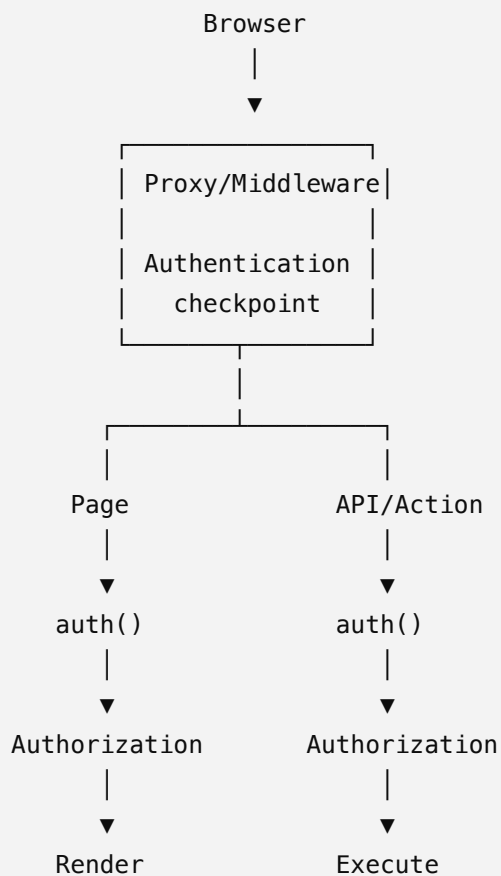
Instead:

Proxy
↓
Protects matched request paths

Server code
↓
Protects sensitive operations

Authorization
↓
Protects specific resources/actions

The architecture to remember



This gives you a very strong interview answer:

Proxy/Middleware can provide centralized, early request-level authentication checks, but sensitive server-side operations should still enforce authentication and authorization at the server boundary.

One thing I want you to notice

Earlier we had:

```
</> TypeScript

const session = await auth();

if (!session) {
  redirect("/login");
}
```



Now Proxy can potentially catch the request **before the page**.

So these aren't competing concepts:

```
Proxy
+
auth()
```



They can work together.

Proxy = early/coarse protection.

auth() = server-side access to the authenticated user/session where the actual operation happens.

Authorization = whether that authenticated user is allowed to perform the specific action.

Next: 7.12.1

We'll go one level deeper and actually trace:

```
GET /dashboard
  ↓
Proxy
  ↓
auth()
  ↓
cookie/session
  ↓
allow vs redirect
  ↓
page.tsx
```



Then we'll answer the tricky interview question:

"If middleware already checks authentication, why should I call `auth()` again?"

That question is 🔥 and the answer ties together almost everything you've learned so far.



For **React**, I'd budget a little more than Next.js because React is the **core interview skill** and you have a 2-year gap to refresh.

Realistic React timeline: 6–8 focused days

Not 2 weeks of textbook study. We can move quickly through things you already know and slow down on the interview-heavy parts.

Day	React focus
Day 1	JSX, components, props, state, events, controlled/uncontrolled components
Day 2	Rendering, reconciliation, Virtual DOM, Fiber, render vs commit, keys ★
Day 3	Hooks — <code>useState</code> , <code>useEffect</code> , <code>useRef</code> , <code>useMemo</code> , <code>useCallback</code> , Context
Day 4	State management + async React — Context, Redux Toolkit/Saga, fetching, race conditions, AbortController
Day 5	Performance — <code>React.memo</code> , referential equality, memoization, re-render debugging, profiling
Day 6	Modern React — Suspense, transitions, <code>useTransition</code> , <code>useDeferredValue</code> , <code>useOptimistic</code> , Actions
Day 7	Architecture — custom hooks, component design, state ownership, folder/feature architecture, forms
Day 8	Testing + React internals + mock interview

★ The areas I want you REALLY strong in

For your mid-level interviews, these matter more than memorizing every React API:

Rendering
↓
Reconciliation
↓
State updates
↓
Hooks
↓
Effects
↓
Re-renders
↓
Performance
↓
Architecture



Especially questions like:

- **Why did this component re-render?**
- Why doesn't `useRef` cause a re-render?
- Why does `useEffect` run again?
- What is a stale closure?
- Why does `useCallback` sometimes do nothing?
- When should you use `useMemo` ?
- How does React reconciliation work?
- Why are keys important?
- Why does changing a key reset state?
- What happens during render vs commit?
- Context vs Redux — when would you choose each?
- How would you optimize a slow React page?

Those are much more valuable than knowing 50 hooks by heart.



So total from where you are now

If we do it efficiently:

Next.js remaining: ~4 days

React revision: ~6–8 days

Combined: ~10–12 focused days

Then I'd spend another **2–3 days** doing:

React + Next.js

↓

Coding questions

↓

Frontend system design

↓

Performance debugging

↓

Mock interviews

↓

🔥 Interview-ready



So roughly **12–15 days for the full React + Next.js refresh**, assuming you're studying seriously each day.

And because you've already learned a lot of the Next.js material, **we don't need to restart anything**. We continue from Route Handlers, finish the high-value Next.js pieces, then switch straight into React revision. 💪



ohh god then when should i start project and when should i start js,redux toolkit and html and css and machine coding round..:(..and most importantly we performance analyzerwhat to do i want to start applying from oct 7th..can i do

YES. ❤️ **You can start applying from October 7th.** But we need to change the strategy **right now.**

The mistake would be thinking:

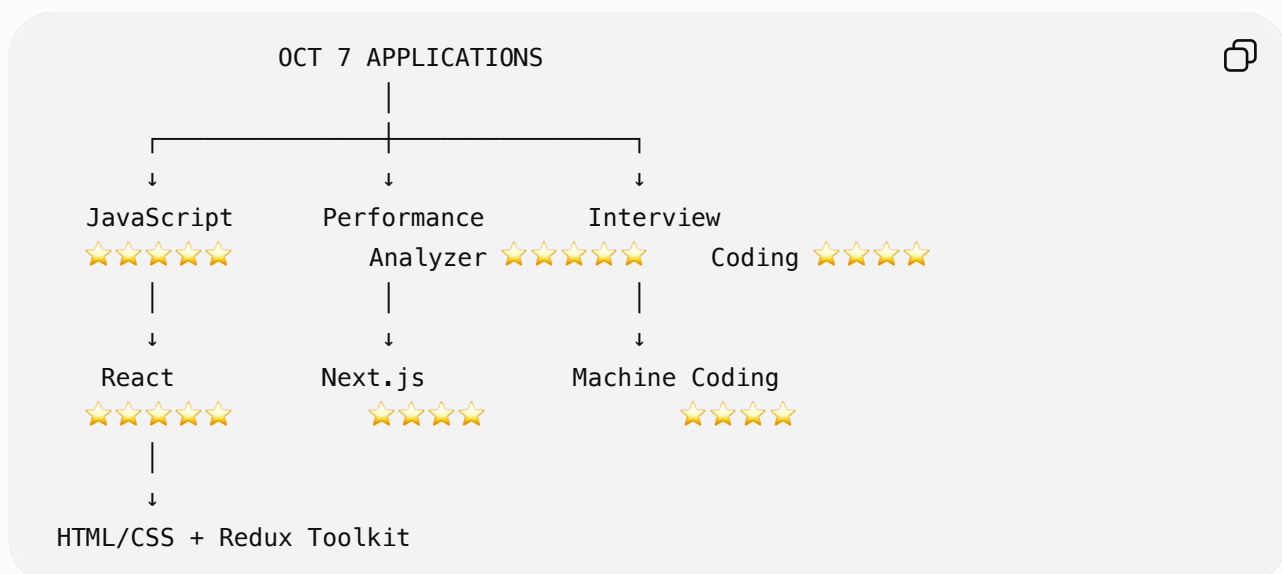
"First finish Next.js → then React → then JS → then Redux → then HTML/CSS → then project → then machine coding → finally apply."

If we do that, **October 7 will arrive before you apply.** 😭

You have **11 days** from Sep 26 → Oct 7. We need to run these **in parallel**, with the **Web Performance Analyzer** as the anchor project.

🚩 Your priority order

For a mid-level frontend role, I'd prioritize:



You do NOT need to "finish" all of these before applying.



Sep 26 → Oct 6

Here's what I'd actually do.

🟣 Sep 26–29 — Finish high-value Next.js + start JS

Next.js

Finish only the **high-value remaining topics**:

- Route Handlers
- API design
- BFF
- Performance
- Deployment basics

Don't spend days on obscure Next.js internals.

JavaScript — START NOW

At the same time:

- Scope
- Hoisting
- Closures
- `this`
- Prototypes
- Objects
- Arrays
- Destructuring
- Promises
- `async/await`
- Event loop
- Microtasks/macrotasks
- Debouncing/throttling
- Error handling

And especially:

★ Your JS interview coding

You've already wanted to practice things like:

- `Promise`
- `Promise.allSettled`
- `retry`
- `timeout`

- concurrency limiter
- rate limiter

Those are **high-value machine/interview exercises**.

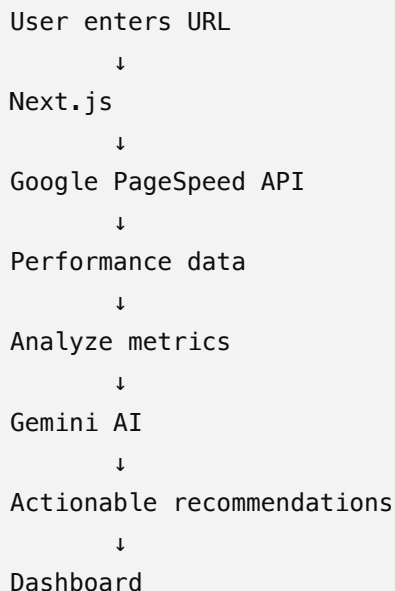
Sep 30–Oct 2 — Web Performance Analyzer becomes priority #1

This is where I want you to **stop studying endlessly and build**.

Your project:

Web Performance Analyzer

You already have the idea:



```
graph TD; A[User enters URL] --> B[Next.js]; B --> C[Google PageSpeed API]; C --> D[Performance data]; D --> E[Analyze metrics]; E --> F[Gemini AI]; F --> G[Actionable recommendations]; G --> H[Dashboard];
```

The flowchart illustrates the process of the Web Performance Analyzer. It starts with 'User enters URL', which leads to 'Next.js'. From 'Next.js', the process continues to 'Google PageSpeed API', then to 'Performance data', followed by 'Analyze metrics', then 'Gemini AI', then 'Actionable recommendations', and finally ends at 'Dashboard'. A copy icon is visible in the top right corner of the flowchart area.

Build the **MVP first**.

MVP

You need:

- URL input
- Analyze button
- API integration
- Loading state
- Error state
- Performance score
- Core Web Vitals
- Basic recommendations

- Clean responsive UI

That's enough to put it on your resume.

Do NOT spend three days making it beautiful before the functionality works.

Oct 3–4 — React + JS + machine coding

Now do your React revision alongside the project.

Focus on:

React

- Rendering
- Reconciliation
- Hooks
 - `useEffect`
 - `useMemo`
 - `useCallback`
 - `useRef`
- Context
- Re-renders
- Performance

Machine coding

Build **one small component per day**.

For example:

Day 1

Search + debounce



Day 2

Infinite scroll / pagination



or

Autocomplete



Don't try to complete 20 machine-coding problems.

Learn the **patterns**.

Oct 5 — Redux + HTML/CSS refresh

You don't need to relearn these from zero.

Redux Toolkit

Focus on:

```
store
slice
reducer
action
dispatch
selector
createAsyncThunk
middleware
```



And be able to explain:

Why Redux when we already have Context?

That's more valuable than memorizing Redux APIs.

HTML/CSS

Refresh:

- Semantic HTML
- Accessibility
- Forms
- Flexbox
- Grid
- Positioning
- Responsive design
- CSS specificity
- Box model
- `display`
- `z-index`
- Media queries

One day is enough for a targeted refresh because you already have professional frontend experience.

Oct 6 — Interview simulation + project polishing

This day is **not another study day**.

Morning

JavaScript interview questions.

Afternoon

Machine coding.

Evening

Your Web Performance Analyzer:

- README
- screenshots
- architecture explanation
- deployment
- GitHub cleanup
- resume bullets
- be able to explain every important technical decision

And most importantly:

You should be able to answer:

"Why did you use Next.js?"

"Why Server Components?"

"Why did you make this component client-side?"

"How did you handle PageSpeed API secrets?"

"How did you handle loading/error states?"

"How does caching work here?"

"How would you improve the performance of this application?"

"How would you scale it?"

That project becomes your **conversation starter in interviews**.



October 7 — START APPLYING

Do not wait until you feel 100% ready.

Start applying.

And here's the important part:

Applying doesn't mean stopping preparation.

From Oct 7 onward:

Morning



Applications / recruiter outreach

Afternoon



JavaScript + React

Evening



Project + machine coding



As interviews start coming in, your preparation becomes **interview-driven**.



And I want to calm one thing down

You just looked at the roadmap and thought:

"Oh god, there are so many things left."

I get why it feels overwhelming. 😓

But look at what you've **already done**.

You have already gone through:

- React experience
- Next.js fundamentals
- Server Components
- App Router
- Data fetching
- Server Actions
- caching architecture
- RSC

- authentication
- authorization
- OAuth
- JWT
- DB sessions
- CSRF
- CORS
- Proxy
- RBAC

You're **not starting from zero**.

The problem now is **sequencing**, not lack of knowledge.

Your new rule

From today:

Don't finish a topic before starting the next one.

We run multiple tracks.

Track A — Interview knowledge

JS → React → Next.js

Track B — Portfolio

Web Performance Analyzer

Track C — Coding

Machine coding + JS coding

Track D — Interview

Mock questions + system design

Track E — Job search

Starts Oct 7

And here's what I would NOT do

- ❌ Spend 8 days "finishing React" before touching the project
- ❌ Spend another week completing every Next.js advanced topic
- ❌ Relearn Redux from beginner level
- ❌ Spend 3 days on HTML/CSS
- ❌ Wait until you feel completely ready to apply
- ❌ Build a gigantic Web Performance Analyzer before applying

Instead:

Sep 26

↓

JS + finish Next.js core

↓

Build Performance Analyzer

↓

React + machine coding

↓

Polish project + resume

↓

🔥 Oct 7 → APPLY

↓

Continue preparation while interviewing



Yes, Oct 7 is achievable. But from now until then, we're in **execution mode**, not "complete every possible syllabus" mode. 💪 🔥

And honestly, **the Web Performance Analyzer should become the center of the next 10 days**, not something we postpone until we've finished every topic.



lets do one thing...we will frst complete next.js topics..then complete web performance analyzer...then react half day j hald fay..till oct 10th we will complete and start applying..??

YES. ❤️ I actually like this plan **better** because it reduces the mental clutter. Instead of juggling 5 things every day, we finish one major block and move on.

And **October 10 is still a realistic application start date** if we keep the scope tight.

🎯 **New plan: Sep 26 → Oct 10**

Sep 26–29



🔥 FINISH NEXT.JS



Sep 30–Oct 5/6



🔥 WEB PERFORMANCE ANALYZER



Oct 6–9



⚙️ REACT REVISION



Oct 10



🚀 START APPLYING



There is one thing I'd change slightly: **don't make React only a half-day total.** 😓

If you mean **half a day per React topic**, that's perfectly fine. We can do a rapid but meaningful revision because you already have professional React experience.

Phase 1 — Finish Next.js

Sep 26–29

We continue exactly from where we stopped:

8. Route Handlers & API Architecture

- Route Handlers
- HTTP methods
- Request / NextResponse
- Dynamic API routes
- Validation
- Error handling
- Status codes
- Route Handler vs Server Action

9. BFF / Backend Architecture

- What BFF means
- Why Next.js can act as a BFF
- Server-to-server requests
- Secrets/API keys
- External APIs

- Your PageSpeed API architecture

10. Performance

- Core Web Vitals
- LCP
- INP
- CLS
- Server vs Client performance
- Images
- Fonts
- Dynamic imports
- Code splitting
- Caching
- Streaming
- Bundle size

11. SEO + Error Handling

- Metadata
- `generateMetadata`
- Sitemap
- Robots
- `notFound()`
- `error.tsx`
- `global-error.tsx`

12. Security + Deployment

- CSP basics
- Environment variables
- Server-only secrets
- Production builds
- Vercel/deployment
- Runtime basics

13. Testing + Production basics

Only interview-level depth.

Then:



Next.js mock interview

If we discover a gap, fix it immediately rather than adding another giant lesson.

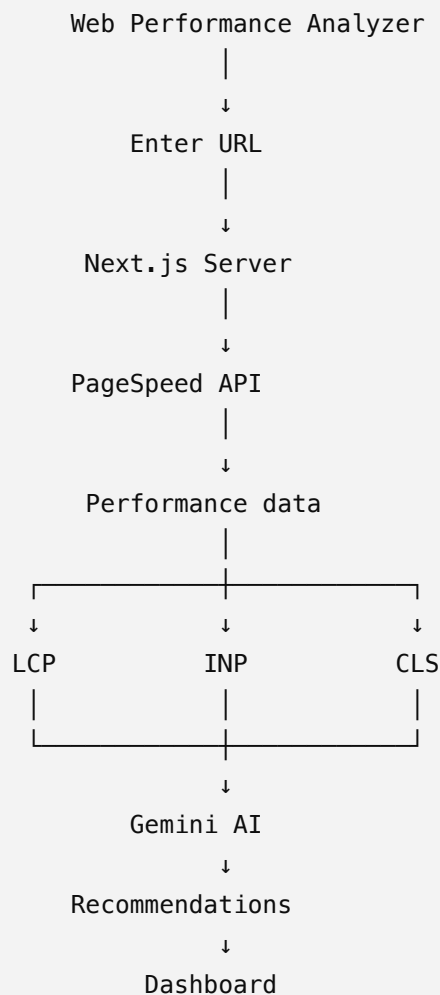
■ Phase 2 — Web Performance Analyzer

Around Sep 30 → Oct 5/6

This becomes priority #1.

We're not going to keep adding features forever.

MVP



We need:

- URL input
- Validation
- Analyze action
- PageSpeed API
- Loading state
- Error state
- Performance score
- Core Web Vitals
- Important metrics

- AI recommendations
- Responsive UI
- Good empty/loading/error states
- Deployment
- README
- Architecture explanation

And then we STOP. 🛑

No:

"Oh, maybe I should add authentication."

"Maybe I should add history."

"Maybe I should add charts everywhere."

"Maybe I should add 17 more features."



Those can become **future enhancements**.

The goal is a project you can confidently discuss in an interview.

Phase 3 — React rapid revision

Oct 6–9

We'll do this **much faster than the original 8-day roadmap** because you already know React.

Half-day blocks

Block 1 — Fundamentals + rendering

- JSX
- Props
- State
- Render cycle
- Reconciliation
- Virtual DOM
- Fiber
- Render vs commit

Block 2 — Hooks

- `useState`
- `useEffect`
- `useRef`
- `useMemo`
- `useCallback`
- Context
- Stale closures

Block 3 — Performance

- Re-renders
- `React.memo`
- Referential equality
- Memoization
- Profiling
- Why components re-render

Block 4 — State + async

- Context vs Redux
- Redux Toolkit basics
- Async state
- Race conditions
- `AbortController`
- Optimistic UI

Block 5 — Modern React

- Suspense
- `useTransition`
- `useDeferredValue`
- `useOptimistic`
- Actions
- Modern form patterns

Block 6 — Architecture + machine coding

- Custom hooks
- Component design
- State ownership
- Reusable components
- Common machine-coding patterns

And we'll sprinkle **JavaScript interview questions** into these sessions rather than waiting until the end.



October 10

Applications start.

Not:

"I've completed 100% of everything."

Instead:

"I have a strong React + Next.js foundation, a real project I can explain deeply, and I'm actively preparing while interviewing."

That's the mindset.

And what about JS, HTML/CSS, Redux?

We do not ignore them.

We'll do targeted revision **after October 10** while applications/interviews are running.

JavaScript = highest priority

We'll cover:

- Closures
- Scope
- Hoisting
- `this`
- Prototypes
- Event loop
- Promises
- `async/await`
- Microtasks/macrotasks
- Debounce/throttle
- Array/object patterns
- Implementations
- DSA/interview coding

HTML/CSS

Rapid revision:

- Semantic HTML

- Accessibility
- Flexbox
- Grid
- Responsive design
- Positioning
- Specificity
- Box model

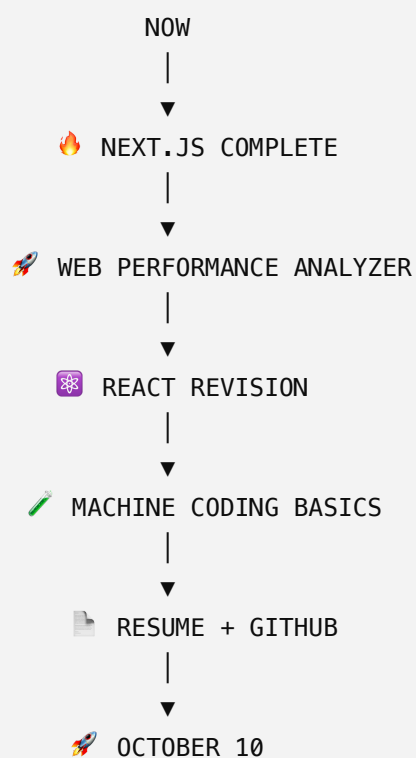
Redux Toolkit

Since you've already used Redux:

- Store
- Slice
- Reducer
- Action
- Dispatch
- Selector
- Async thunks
- Middleware
- Architecture

We don't need to spend a week on it.

❤️ So yes — let's lock this plan



APPLY APPLY APPLY



Continue JS / Redux /
HTML-CSS alongside
interview preparation

I think this is much less overwhelming and much more executable. 🙌

So in the **new Next.js chat**, we start with **8.1 Route Handlers** and keep moving until Next.js is finished. No detours into React or JS until that block is done.



ChatGPT can make mistakes. Check important info.